

Unity C# R&D Programmer Portfolio Guide

지원자: 오지은

지원 직무: 마비노기 모바일 연구개발 프로그래머

포트폴리오 GitHub: <https://github.com/jieundino/GameProgrammer-Portfolio>

1. 포트폴리오 개요

본 포트폴리오는 Unity C# 기반 프로젝트에서 **기획 요구사항을 기능 구조로 변환하고, 작은 단위로 구현·검증하며, 런타임 문제를 분석하고 문서화한 경험**을 정리한 자료입니다.

공동개발 프로젝트의 전체 코드를 제출하는 대신 지원자가 직접 설계·구현·수정한 코드와 다음 근거를 함께 수록했습니다.

- 기능별 설계 의도와 처리 흐름
- 코드 소유 범위와 공동개발 기여 비중
- 실제 런타임 이슈의 재현 조건과 수정 결과
- 구조적 한계와 후속 개선 방향
- GitHub 코드 샘플과 프로젝트 결과물

핵심 역량

- 요구사항 분석 및 기능 단위 분해
- 스킬 설정 데이터와 Runtime State 분리
- 실행 성공 조건과 실패 경로를 포함한 기능 구조 설계
- 데이터 기반 콘텐츠 구조와 공통 처리·예외 규칙 분리
- 상태 관리와 실패 대응 로직 구현
- 신규 기능 프로토타이핑 및 실제 환경 검증
- 출시·전시 환경의 런타임 이슈 대응
- 기술적 의사결정과 QA 결과 문서화

2. 연구개발 프로그래머 직무 이해

연구개발 업무	본 포트폴리오의 근거
신규 기능 구조화 및 구현	Skill Data / Runtime / Validation / Execution 구조, CSV 이벤트 파이프라인, SFX 채널 정책, 미니게임 프로토타입
기존 코드 유지보수	공통 처리와 예외 규칙 분리, 저장 데이터 복구, 상태 동기화
런타임 이슈 대응	입력 중복, 비정상 드롭, 부분 성공 상태, 직렬화 오류, 씬 경계 참조, 오디오 자원 누적 분석
기술 탐색과 적용	ScriptableObject Runtime 분리, NonAlloc Physics Query, Atomic Write, Retry Queue, ScriptableObject Command, Voice Stealing
검증	스킬 성공·실패·경계 조건 검증, 출시·전시 빌드 QA, Android 테스트, 사용자 테스트, 로그 기반 확인
문서화 및 공유	Architecture & Feature Verification, Technical Notes, Runtime Issue Analysis, Notion 시퀀스 문서, 학술 논문

연구개발 프로그래머는 단순히 새로운 기술을 조사하는 데 그치지 않고, 게임의 사용자 경험을 확장할 기능을 탐색하고 실제 코드베이스 안에서 검증 가능한 형태로 구현해야 한다고 이해하고 있습니다.

라이브 게임에서는 신규 기능이 기존 상태와 충돌하지 않아야 하며, 구현 이후에는 발생한 문제의 재현 조건과 영향 범위를 좁혀 서비스 안정성을 유지해야 합니다. 또한 기능의 목적, 구조, 한계와 테스트 결과를 문서화해 다른 구성원이 이해하고 확장할 수 있어야 합니다.

3. 제출 자료 구성

```
00_README.md
01_Portfolio_Guide.pdf
02_Project_Contribution.md
03_SourceCode_Index.md

04_Technical_Notes/
├─ 00_UnitySkillLogicPrototype_Architecture_Verification.md
├─ 01_FourFootsteps_CSV_Event_System.md
├─ 02_FourFootsteps_SaveLoad_System.md
├─ 03_Fate_ActionPointManager.md
├─ 04_Fate_SFX_Priority_Channel.md
└─ 05_HeyCheese_Minigame_Logic.md

05_Runtime_Issue_Analysis/
└─ Runtime_Issue_Analysis.md

06_SourceCode/
├─ 00_UnitySkillLogicPrototype/
├─ 01_FourFootsteps_DataDrivenSystems/
```

└─ 02_Fate_RuntimeSystems/
└─ 03_HeyCheese_MinigamePrototypes/

06_SourceCode에는 지원자가 직접 작성하거나 직접 수정한 파일만 포함했습니다. 원본 프로젝트 전체가 아니므로 일부 Scene, Prefab, Manager, 데이터 파일과 외부 패키지는 생략되어 있습니다.

00_UnitySkillLogicPrototype은 지원자가 단독으로 설계·구현한 개인 기술 프로토타입이며, 나머지 폴더는 공동개발 프로젝트에서 직접 작성·수정한 코드를 선별한 자료입니다.

4. 프로젝트 요약

프로젝트	역할	핵심 구현	연구개발 관점의 의미
Unity Skill Logic Prototype	Solo Developer / System Designer	Skill Data·Runtime 분리, 실행 검증, Normal·DoT·Area 스킬, 전투 피드백	일반적인 스킬 명세를 실행 구조로 변환하고 성공·실패·경계 조건을 검증
네 발자국	Main Client Programmer	CSV 이벤트, Event/Result Pipeline, Save/Load, Log Retry	기획 데이터를 실행 구조로 변환하고 실패 대응까지 연결
필연과 우연	Client / System Programmer	ActionPoint 상태 흐름, 퍼즐 입력 판정, SFX 채널 정책	실제 플레이 예외를 재현하고 공통 규칙과 특수 규칙 분리
헤이 치즈!	Mini-game Client Programmer	4종 미니게임, 터치 입력, 시간 흐름, 상태 피드백	기능을 작은 단위로 구현하고 Android·사용자 환경에서 검증

5. Unity Skill Logic Prototype

5.1 프로젝트 개요

Unity Skill Logic Prototype은 일반적인 전투 스킬 명세를 설정 데이터, 플레이 중 상태, 실행 조건과 효과 적용 단계로 분해하고, Normal·DoT·Area 스킬의 동작을 검증한 Unity C# 개인 기술 프로토타입입니다.

완성형 전투 시스템 제작보다 서로 다른 스킬 규칙을 공통 실행 흐름 안에서 처리하고, 실패 조건에서도 상태 일관성을 유지할 수 있는지 확인하는 데 목적을 두었습니다.

- 담당 기간: 2026.07
- 개발 형태: 개인 기술 프로토타입
- 주요 기술: Unity, C#, ScriptableObject, Coroutine, Physics Query, UGUI
- 담당 범위: 시스템 구조 설계, C# 구현, 기능 검증, 기술 문서 작성
- 결과: Normal·DoT·Area 스킬, Cooldown·Health UI, 기능 검증 영상 및 소스코드 문서화

5.2 요구사항을 실행 단계로 분해

스킬 사용 과정을 다음 단계로 구분했습니다.

```
Skill Data
→ Runtime State
→ Activation Validation
→ Target Search / Range Validation
→ Skill Effect Execution
→ Cooldown / Animation / UI Feedback
```

- **SOSkill**: 피해량, 쿨타임, 사거리, DoT와 Area 설정 데이터
- **SkillRuntime**: 슬롯별 남은 쿨타임과 사용 가능 상태
- **PlayerSkillController**: 입력, 슬롯 조회, 사용 상태 검사와 성공 시 쿨타임 적용
- **SkillExecutor**: 대상 탐색, 사거리 판정과 SkillType별 효과 실행
- **EnemyHealth / DotStatusEffect**: 체력, 사망 상태와 시간 기반 지속 피해
- Presenter / SkillButton: 실제 Runtime 상태를 화면 피드백으로 변환

연구개발 관점

기능 명세를 하나의 클래스에 직접 구현하지 않고, 변경 빈도와 실행 시점에 따라 설정 데이터, 런타임 상태, 검증과 효과 실행 책임으로 분리했습니다.

5.3 핵심 구현 1 — Skill Data와 Runtime State 분리

피해량, 사거리와 쿨타임 설정은 ScriptableObject Asset에 저장하되, 플레이 중 변하는 남은 쿨타임은 별도의 **SkillRuntime** 객체에서 관리했습니다.

동일한 Skill Asset을 여러 슬롯이나 캐릭터가 참조할 때 ScriptableObject에 Runtime 값을 저장하면 상태가 공유될 수 있습니다. 각 슬롯이 독립적인 Runtime 객체를 생성하도록 구성해 설정 데이터와 플레이 상태가 섞이지 않도록 했습니다.

연구개발 관점

- 데이터 재사용과 플레이 상태 독립성을 함께 유지했습니다.
- Skill UI도 별도 타이머가 아니라 `SkillRuntime.CooldownRatio` 를 사용해 실제 사용 가능 상태와 표시 기준을 통일했습니다.

5.4 핵심 구현 2 — 실행 성공 기준과 상태 변화 순서

스킬 입력 직후 쿨타임을 적용하지 않고, 실제 실행 결과가 성공인 경우에만 쿨타임과 애니메이션을 시작했습니다.

```
스킬 입력
→ Runtime 사용 가능 상태 확인
→ 대상 탐색 및 실행 조건 검증
→ 효과 적용 성공 여부 반환
  └─ 실패: 상태 변화 없음
  └─ 성공: Cooldown + Animation + Feedback
```

단일 대상 스킬은 유효한 대상이 사거리 안에 있어야 성공합니다. 반면 자기 중심 Area Skill은 적중 대상이 없어도 스킬 자체가 발동하므로 성공으로 처리합니다.

연구개발 관점

모든 스킬에 동일한 성공 기준을 적용하지 않고, Target-dependent Skill과 Self-activated Skill의 규칙을 분리했습니다. 실행 실패와 적중 0명을 구분해 이후 상태 변화의 기준을 명확히 했습니다.

5.5 핵심 구현 3 — 대상 탐색과 중복 피해 방지

단일 대상 스킬은 `Physics.OverlapSphereNonAlloc` 으로 사거리 안의 적을 탐색하고, 사망하지 않은 대상 중 제곱 거리가 가장 짧은 대상을 선택합니다.

범위 공격에서는 Physics 결과가 Collider 단위로 반환되기 때문에 하나의 Enemy가 여러 Collider를 보유하면 중복 피해가 발생할 수 있습니다. `HashSet<EnemyHealth>` 를 사용해 Collider 결과를 실제 전투 대상 단위로 변환하고, Enemy당 한 번만 피해를 적용했습니다.

연구개발 관점

- 재사용 Buffer를 사용해 반복 Physics Query의 할당을 줄였습니다.

- 엔진 API가 반환하는 단위와 게임 로직에서 필요한 대상 단위가 다르다는 점을 확인하고 변환 단계를 추가했습니다.

5.6 기능 검증

검증 시나리오	확인 결과
사용 가능한 단일 대상 스킬	피해 적용 후 쿨타임 시작
쿨타임 중 재사용	실행 차단
단일 대상 없음	실행 실패, 쿨타임 미적용
대상이 사거리 밖	실행 실패, 쿨타임 미적용
사망한 대상	신규 Target 후보에서 제외
DoT 재적용	기존 Coroutine을 종료하고 지속 시간 갱신
Area Skill 적용 0명	스킬 발동 성공, 쿨타임 적용
하나의 Enemy에 여러 Collider	Enemy 기준 피해 1회
적 체력 변화와 사망	Health Bar 갱신 및 비활성화
스킬 성공	대상 방향 회전과 Animation 실행

연구개발 관점

정상 실행뿐 아니라 대상 없음, 사거리 초과, 사망 대상, 다중 Collider와 재적용처럼 상태 불일치가 발생할 수 있는 조건을 별도 시나리오로 정의하고 확인했습니다.

5.7 현재 한계와 후속 개선

- 새로운 SkillType 추가 시 `SkillExecutor` 의 switch문 수정 필요
- Targeting과 Effect Execution이 한 클래스에 함께 존재
- DoT는 Refresh 정책만 지원
- Buff / Debuff / Stack, 자원 소비, 캐스팅, Line of Sight 미구현
- UI는 Runtime 상태를 매 Frame 조회
- 수동 기능 검증 중심이며 자동화된 PlayMode Test 미구현

후속 단계에서는 Targeting Rule과 Skill Effect를 독립 실행 단위로 분리하고, 여러 Effect를 조합할 수 있는 구조와 Unity Test Framework 기반 자동 검증을 적용할 수 있습니다.

6. 네 발자국

6.1 프로젝트 개요

네 발자국은 인간과 고양이의 시점을 오가며 플레이어의 선택과 조사 결과에 따라 상태값과 엔딩이 달라지는 Unity 기반 2D 내러티브 어드벤처 게임입니다.

지원자는 메인 클라이언트 프로그래머로서 대화·선택지·이벤트·결과 실행·저장·로그가 동일한 게임 상태를 기준으로 동작하도록 클라이언트 구조를 설계했습니다.

- 담당 기간: 2025.03 ~ 2026.02
- 개발 형태: 4인 공동개발
- 주요 기술: Unity, C#, CSV, Dictionary, Newtonsoft.Json, UnityWebRequest, PlayerPrefs
- 결과: STOVE 출시, 학술발표대회 및 학술지 논문 제1저자

6.2 핵심 구현 1 — 데이터 기반 Event / Result Pipeline

대화와 선택지, 이벤트 조건과 결과를 코드에 직접 작성하면 기획 변경마다 실행 코드를 수정해야 하고, 이벤트 수가 늘어날수록 조건문과 책임이 한 시스템에 집중될 수 있었습니다.

이를 해결하기 위해 데이터를 CSV와 ID 기반으로 분리했습니다.

플레이어 입력 또는 이벤트 요청
→ EventManager 조건 판정
→ 실행 가능 여부 확인
→ Instant / Sequential 실행 모드 선택
→ ResultManager 결과 실행
→ 상태값·대사·연출 갱신

- **EventManager**: 현재 상태와 AND/OR 조건을 비교해 이벤트 실행 가능 여부 판정
- **ResultManager**: Result ID에 따라 변수 변경, 대사, 페이드, 씬 이동, 오브젝트 동작 실행
- **Instant / Sequential**: 독립 실행 결과와 순서 의존 결과를 분리
- **Dialogue Queue**: 여러 대사 요청이 동시에 발생해도 출력 순서 유지

연구개발 관점

- 기획 요구사항을 데이터 정의와 실행기로 분해했습니다.
- 콘텐츠 수정 시 코드 변경 범위를 줄였습니다.

- 조건 판정과 결과 적용의 책임을 분리했습니다.
- Result 유형 증가 시 분기 규모가 커지는 한계를 확인하고 Command 구조와 데이터 검증 도구를 후속 개선 방향으로 정리했습니다.

6.3 핵심 구현 2 — Save Reliability

저장 도중 강제 종료가 발생하면 원본 파일이 손상될 수 있고, `Dictionary<string, object>` 와 복합 타입은 역직렬화 과정에서 타입 정보가 달라질 수 있었습니다.

- 임시 파일에 먼저 기록한 뒤 원본과 교체하는 Atomic Write 적용
- 기존 파일을 `.bak` 으로 백업
- `File.Replace` 실패 시 Delete/Move 방식으로 fallback
- 변수 값과 타입 정보를 분리 저장
- 레거시 데이터와 일부 손상 데이터 복구 경로 구성
- 내부 상태 복원 이후 UI와 현재 이벤트 화면 갱신

연구개발 관점

정상 경로뿐 아니라 저장 실패, 타입 불일치, 구버전 데이터라는 실패 경로를 정의하고 전체 Save/Load 실패로 확산되지 않도록 방어했습니다.

6.4 핵심 구현 3 — Log Tracking & Retry

사용자 테스트에서는 선택, 책임지수, 퍼즐 상태와 도달 엔딩을 안정적으로 수집해야 했습니다.

- RunID와 eventId를 이용한 로그 식별
- 클라이언트와 서버 양쪽의 중복 전송 방지
- UnityWebRequest 기반 비동기 전송
- 네트워크 실패 시 PlayerPrefs Retry Queue 저장
- 성공 응답을 받은 로그만 Queue에서 제거
- 게임 상태 로드 시점과 분리하기 위해 저장 파일에서 필요한 값 추출

연구개발 관점

기능 실행 결과를 식별 가능한 데이터로 기록하고, 실패한 요청을 보존·재처리하는 구조를 구현했습니다. 테스트 데이터 수집 이후 결과를 분석하고 논문으로 문서화했습니다.

7. 필연과 우연

7.1 프로젝트 개요

필연과 우연은 퍼즐, 단서 획득, 행동력, 날짜 진행, 사운드 연출과 멀티엔딩이 연결된 Unity 기반 2D 방탈출 어드벤처 게임입니다.

지원자는 클라이언트 및 시스템 프로그래머로서 플레이어 입력을 조건 판정, 상태 변화와 연출로 연결하는 기능을 구현하고, 출시·전시 환경에서 발생한 런타임 문제를 개선했습니다.

- 담당 기간: 2024.03 ~ 2026.01
- 개발 형태: 6인 공동개발
- 주요 기술: Unity, C#, UI Event System, Coroutine, AudioSource
- 결과: STOVE·App Store 출시, BIC·Beaver Rocks 전시

7.2 핵심 구현 1 — ActionPointManager 상태 흐름

행동력 감소는 하트 UI, 날짜 전환, 시계·페이지 연출, BGM과 후속 이벤트에 연결되어 있었습니다. 또한 Room마다 자원 소모 및 회복 특정 이벤트와 날짜 전환 규칙이 달랐습니다.

상호작용 요청

- 처리 가능 상태 확인
- 행동력·UI 상태 검증
- 행동력 차감
- 하트 UI 갱신
- 자원 소진 여부 판정
- 날짜 전환 연출
- 후속 이벤트 실행

- 공통 처리 흐름은 추상 클래스 `ActionPointManager`에 정의
- Room별 규칙은 하위 클래스에 분리
- 이벤트 처리 중 추가 입력 차단
- 내부 행동력과 UI 상태를 함께 검사해 중복 차감 방지
- Coroutine으로 시간 기반 연출 순서 관리

연구개발 관점

공통 규칙과 콘텐츠별 예외를 분리해 변경 영향 범위를 줄였습니다. 동시에 UI 상태를 검증 기준으로 사용하는 현재 구조의 결합도를 확인하고, 단일 상태 모델과 이벤트 기반 UI 갱신을 개선 방향으로 정리했습니다.

7.3 핵심 구현 2 — 퍼즐 입력 유효성 판정

드래그 종료 위치만으로 배치하면 DropZone 경계와 화면 밖 방향의 입력에서 UI와 내부 배치 정보가 어긋날 수 있었습니다.

- `RectTransformUtility` 기반 좌표 변환
- 최근접 DropZone 탐색
- 비즈 간 행·순서 제약 검사
- 유효하지 않은 위치는 자동 보정 또는 원래 위치 복귀
- Dictionary 기반 정답 상태 확인

연구개발 관점

사용자가 예상과 다르게 입력하는 경계 상황을 재현하고, 거리 판정과 규칙 검사를 단계적으로 적용해 내부 상태의 일관성을 유지했습니다.

7.4 핵심 구현 3 — SFX Priority Channel System

반복 클릭음이 AudioSource 채널을 점유하면 날짜 전환과 같은 중요한 연출음이 유실될 수 있었습니다.

- `Low / High` 2단계 우선순위 정책
- High 전용 예약 채널 구성
- Low 요청의 예약 채널 접근 제한
- Round-robin 방식의 빈 채널 탐색
- 채널 포화 시 High 요청의 Voice Stealing
- `Time.unscaledTime` 기반 클릭음 Debounce

연구개발 관점

한정된 런타임 자원을 우선순위 정책에 따라 할당하고, 포화 상황의 fallback을 구현했습니다. 현재 구현은 High 효과음이 일반 영역에서 재생될 수 있어 Voice Stealing 대상의 실제 우선순위를 별도로 추적하지 못한다는 한계가 있으며, 채널 상태 모델 추가를 개선 방향으로 정리했습니다.

8. 헤이 치즈!

8.1 프로젝트 개요

헤이 치즈!는 감정 표현과 사회적 상호작용을 주제로 한 Android 교육용 인터랙티브 콘텐츠입니다.

지원자는 미니게임 클라이언트 프로그래머로서 서로 다른 규칙을 가진 4종 미니게임의 입력, 진행 상태, 완료 조건과 피드백을 구현했습니다.

- 담당 기간: 2025.03 ~ 2025.07
- 개발 형태: 4인 공동개발
- 주요 기술: Unity, C#, Touch Input, Coroutine, UGUI
- 결과: 사용자 테스트용 Android 빌드, JCCT 논문 제2저자

8.2 기능 프로토타이핑

- 숨바꼭질, 음식 선택/식탁 정리, 달리기, 춤추기 미니게임 구현
- 공통 시작·완료·재시작·복귀 흐름 구성
- 미니게임별 입력 방식과 완료 조건 분리
- Coroutine 기반 카운트다운, 제한 시간, 이동과 단계 진행
- Slider, Text, Sprite를 활용한 상태 피드백

미니게임 시작
→ 상태 초기화
→ 사용자 입력
→ 진행 조건 판정
→ 상태·UI 갱신
→ 완료 조건 확인
→ 결과 피드백 및 메인 흐름 복귀

8.3 실제 환경 검증

Android 테스트 빌드에서 터치 입력, 화면 피드백, 완료 조건과 메인 스토리 복귀 흐름을 확인했습니다. 일반 초등학생 30명의 파일럿 테스트 결과를 바탕으로 난이도와 조작 흐름을 검토했습니다.

연구개발 관점

서로 다른 규칙을 가진 기능을 작은 단위로 구현하고, 공통 흐름과 개별 규칙을 분리한 뒤 실제 기기와 사용자 환경에서 검증했습니다. 테스트 대상의 한계를 문서화하고 후속 사용자 검증 과제로 분리했습니다.

9. Runtime Issue Analysis

본 포트폴리오에는 실제 개발과 테스트 과정에서 재현한 16건의 런타임 이슈를 별도 문서로 정리했습니다.

대표 사례는 다음과 같습니다.

문제	원인 분석	수정 방향
행동력 중복 차감	처리 중 추가 입력과 상태 검증 부족	실행 상태·행동력·UI 검증 후 요청 수락
MemoryPuzzleStates 직렬화 실패	복합 타입이 유효한 JSON이 아닌 문자열로 저장	전용 직렬화 타입과 fallback 복구 추가
UnityEvent 씬 경계 참조 불안정	Inspector 참조가 씬 객체 기준으로 직렬화	ScriptableObject Command로 실행 시점 서비스 탐색
오디오 메모리 누적	이전 코루틴과 AudioClip 참조가 씬 전환 후 잔존	전환 전 코루틴 중단, Stop, clip 참조 해제

각 항목은 영향, 재현 조건, 원인, 수정, 확인 결과와 재발 방지 관점으로 구성했습니다.

10. 현재 구조의 한계와 후속 개선 방향

현재 구조	확인한 한계	후속 개선 방향
SkillType switch	스킬 유형 증가 시 <code>SkillExecutor</code> 분기 규모 확대	Targeting Rule과 Skill Effect를 Strategy 단위로 분리
Skill Runtime UI Polling	SkillButton이 매 Frame Runtime 상태 조회	상태 변경 이벤트 기반 UI 갱신 비교
수동 스킬 검증	반복 검증과 회귀 확인 비용 증가	Unity Test Framework 기반 EditMode / PlayMode Test
CSV Event/Result	잘못된 ID와 참조 누락을 런타임 전에 확인하기 어려움	Unity Editor 기반 Event Data Validator
ResultManager switch	Result 유형 증가 시 분기 규모 확대	Command / Strategy 기반 실행 클래스 분리
ActionPoint 상속 구조	Room 증가 시 하위 클래스와 재정적 범위 증가	데이터·Strategy 기반 규칙 구성

현재 구조	확인한 한계	후속 개선 방향
행동력·UI 교차 검증	UI와 도메인 상태 결합	단일 상태 모델과 이벤트 기반 UI 구축
SFX Voice Stealing	일반 영역에 재생 중인 실제 Priority 추적 부족	ChannelState에 현재 Priority와 Clip 기록
Coroutine 시퀀스	중단·취소·인터럽트 정책이 분산될 수 있음	명시적 상태 머신과 Cancellation 정책

완성된 구조를 과장하기보다, 실제 코드가 가진 제약을 확인하고 다음 구현·검증 과제로 구체화했습니다.

11. 마무리

본 포트폴리오는 Unity C# 프로젝트에서 다음 과정을 수행한 경험을 중심으로 구성했습니다.

요구사항 파악
 → 기능·데이터·Runtime State 구조 설계
 → 작은 단위 구현
 → 성공·실패·경계 조건 검증
 → 실제 환경 검증
 → 런타임 문제 재현·수정
 → 결과와 한계 문서화

개인 기술 프로토타입에서는 스킬 명세를 실행 구조로 변환하고 기능별 성공 조건과 예외 상황을 검증했습니다. 공동개발 프로젝트에서는 지원자가 직접 작성하거나 수정한 코드만 선별했으며, 프로젝트별 담당 비중과 협업 경계는 `02_Project_Contribution.md`에 별도로 정리했습니다.

마비노기 모바일의 기존 코드와 사용자 경험을 정확히 이해하고, 신규 기능을 검증 가능한 형태로 구현하며, 라이브 환경의 문제를 근거 중심으로 분석하고 공유하는 연구개발 프로그래머로 기여하겠습니다.